

ALB 演習 手順書 (ALB-1 / ALB-2 / ALB-3)

前提

- OS : Amazon Linux 2023
- Web : Apache httpd
- Web サーバ : 2台 (web-1 / web-2)
- ALB : インターネット向け (Application Load Balancer)
- リスナー : HTTP 80

■ ALB-1 : 2台 均等振り分け構成

1. 各 EC2 で httpd を構築する (web-1 / web-2 両方)

この工程でしていること

- Web サーバ (httpd) を起動し、ALB の転送先として利用できる状態にする
- 各サーバの応答が識別できるよう、index.html の内容を分ける

実行コマンド

```
dnf install -y httpd
```

```
systemctl enable httpd
```

```
systemctl start httpd
```

(web-1)

```
echo "This is web-1" > /var/www/html/index.html
```

(web-2)

```
echo "This is web-2" > /var/www/html/index.html
```

コマンドの意味

コマンド	意味
dnf install	パッケージをインストール
systemctl enable	自動起動を有効化
systemctl start	サービス起動
echo >	ファイルに内容を書き込む

確認方法とOKの目安

curl http://localhost/

- web-1 では **This is web-1**
- web-2 では **This is web-2** が返ること

2. ターゲットグループ (TG) を作成する

この工程でしていること

- ALB が転送する「宛先グループ」を作成する
- ヘルスチェックで生存判定を行えるようにする

実行内容 (AWS コンソール)

- タイプ：インスタンス
- プロトコル：HTTP
- ポート：80
- ヘルスチェックパス：/
- ターゲット：web-1 / web-2 を登録

確認方法とOKの目安

- 登録した web-1 / web-2 が **healthy** になること

3. ALB を作成する

この工程でしていること

- インターネットからアクセスできるロードバランサを作成する
- リスナー (80) から TG へ転送する

実行内容 (AWS コンソール)

- 種別：Application Load Balancer
- スキーム：インターネット向け
- サブネット：パブリックサブネット 2つ
- リスナー：HTTP 80
- デフォルトアクション：作成した TG へ転送

セキュリティグループ（重要）

ALB 側 SG

- Inbound : TCP 80
- Source : 0.0.0.0/0

Web 側 SG

- Inbound : TCP 80
- Source : ALB の SG

確認方法とOKの目安

- ALB が Active になっている
- リスナーが TG に紐づいている
- TG のターゲットが healthy

4. 均等振り分けを確認する

この工程でしていること

- ALB 経由アクセスが web-1 / web-2 に分散されることを確認する

実行コマンド

```
for i in {1..10}; do curl http://ALB-DNS; done
```

コマンドの意味

- for : 繰り返し
- curl : HTTPアクセス確認
- ALB-DNS : ALB の DNS 名

確認方法とOKの目安

- 応答が **This is web-1** と **This is web-2** に交互（または概ね均等）に変わる

サーバ側ログ確認（任意）

（web-1 / web-2 それぞれで）

```
tail -f /var/log/httpd/access_log
```

- 両サーバの access_log にアクセスが記録されること

■ ALB-2：パスベースルーティング

1. 各サーバにコンテンツを作成する

この工程でしていること

- パスごとに振り分けるため、各サーバに固有の URL を用意する

実行コマンド

(web-1)

```
mkdir -p /var/www/html/saba
```

```
echo "saba server" > /var/www/html/saba/saba.html
```

(web-2)

```
mkdir -p /var/www/html/miso
```

```
echo "miso server" > /var/www/html/miso/miso.html
```

コマンドの意味

- `mkdir -p`：必要な親ディレクトリごと作成
- `echo >`：ファイル作成＋内容書き込み

確認方法とOKの目安

(web-1)

```
curl http://localhost/saba/saba.html
```

- `saba server` が返る

(web-2)

```
curl http://localhost/miso/miso.html
```

- **miso server** が返る
-

2. ターゲットグループを分割する

この工程でしていること

- パスごとに転送先 TG を分ける (L7 振り分け)
- ヘルスチェックも各ページに合わせる

実行内容 (AWS コンソール)

TG-saba

- web-1 のみ登録
- ヘルスチェック: **/saba/saba.html**

TG-miso

- web-2 のみ登録
- ヘルスチェック: **/miso/miso.html**

確認方法とOKの目安

- TG-saba で web-1 が healthy
 - TG-miso で web-2 が healthy
-

3. ALB のリスナールールを設定する

この工程でしていること

- URL のパスにより、転送先 TG を切り替える

実行内容 (AWS コンソール: リスナー 80 のルール)

ルール1

- IF Path is **/saba/***
- THEN Forward to **TG-saba**

ルール2

- IF Path is **/miso/***
- THEN Forward to **TG-miso**

確認方法とOKの目安

```
curl http://ALB-DNS/saba/saba.html
```

```
curl http://ALB-DNS/miso/miso.html
```

- /saba は **saba server**
- /miso は **miso server** が返ること

■ ALB-3：ヘルスチェック専用ページ + ログ分離

1. ヘルスチェック専用ページを作成する（両サーバ）

この工程でしていること

- ALB のヘルスチェック専用 URL を用意する
- 通常アクセスとは分離できるようにする

実行コマンド

```
echo OK > /var/www/html/are_you_ok
```

```
chmod 644 /var/www/html/are_you_ok
```

コマンドの意味

- echo OK：固定応答のファイルを作る
- chmod 644：読み取り可能にする

確認方法とOKの目安

```
curl http://localhost/are_you_ok
```

- **OK** が返る

2. ターゲットグループのヘルスチェックを変更する

この工程でしていること

- ヘルスチェック用 URL を **/are_you_ok** に統一する
- 「アプリが壊れてもヘルスチェックが通る」状態を避ける（専用判定）

実行内容（AWS コンソール）

- ヘルスチェックパス：[/are_you_ok](#)

確認方法とOKの目安

- 各 TG のターゲットが healthy を維持する
-

3. Apache のログ分離設定を行う（両サーバ）

この工程でしていること

- ヘルスチェックアクセスだけ別ログに出す
- 通常アクセスログを見やすくする（運用改善）

3-1. 通常ログ設定をコメントアウトする

実行コマンド

```
vi /etc/httpd/conf/httpd.conf
```

変更内容

```
#CustomLog "logs/access_log" combined
```

確認方法とOKの目安

- 通常ログ（access_log）設定が無効化されている

3-2. 分離用設定ファイルを作成する

実行コマンド

```
vi /etc/httpd/conf.d/healthcheck-log.conf
```

設定内容

```
SetEnvIf Request_URI "^/are_you_ok/?$" is_healthcheck

CustomLog logs/healthcheck_access_log combined env=is_healthcheck
CustomLog logs/access_log combined env=!is_healthcheck
```

設定の意味

設定	意味
SetEnvIf	条件一致で環境変数付与
env=is_healthcheck	ヘルスチェックのみ記録
env=!is_healthcheck	通常アクセスのみ記録

3-3. 設定反映

実行コマンド

```
apachectl configtest
```

```
systemctl restart httpd
```

コマンドの意味

- apachectl configtest : 構文チェック
- systemctl restart : httpd 再起動

確認方法とOKの目安

- configtest が Syntax OK
- httpd が active (running)

4. ログ分離の確認

この工程でしていること

- ヘルスチェックが専用ログにだけ出ることを確認する
- 通常アクセスが通常ログにだけ出ることを確認する

実行コマンド

```
tail -f /var/log/httpd/healthcheck_access_log
```

```
tail -f /var/log/httpd/access_log
```

確認方法とOKの目安

- `/are_you_ok` へのアクセスは healthcheck_access_log のみに出る

- 通常アクセスは access_log のみに出る
 - User-Agent が **ELB-HealthChecker/2.0** の行が専用ログに出る
-

■ 完成構成まとめ

実現できたこと

- ALB-1：2台へ均等に負荷分散
- ALB-2：パス (/saba /miso) で L7 振り分け
- ALB-3：ヘルスチェック専用 URL + ログ分離（運用最適化）

最終状態

- 負荷分散：OK
- L7 ルーティング：OK
- ヘルスチェック運用：OK
- ログの可観測性向上：OK